

Redes Neuronales

U3 Redes Neuronales Multicapa

Héctor Maravillo

Section 1

The XOR Problem

The XOR Problem

Boolean functions can be interpreted as **classification tasks**. Indeed, depending on the values of the input binary data $\mathbf{x} = (x_1, x_2, \dots, x_l)^T$, the output is either 0 or 1, and x is classified into one of the two classes A ($y = 1$) or B ($y = 0$).

In the **XOR Problem** no single straight line exist that separates the two classes. In contrast, the other two Boolean functions, **AND** and **OR**, are **linearly separable**.

The XOR Problem

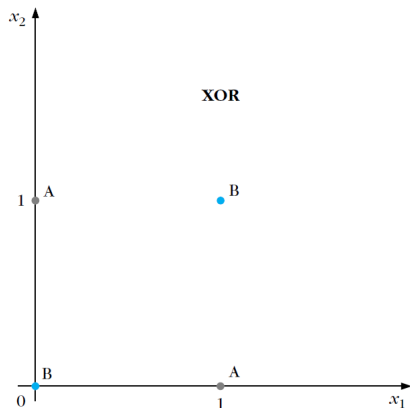


Table 4.1 Truth Table for the XOR Problem

x_1	x_2	XOR	Class
0	0	0	B
0	1	1	A
1	0	1	A
1	1	0	B

Figure: Classes A and B for the XOR problem.

The two-layer perceptron

To **separate the two classes** A and B , two straight lines can be used instead of one. The Figure shows two such possible lines

$$g_1(\mathbf{x}) = g_2(\mathbf{x}) = 0.$$

Class A is to the right (+) of $g_1(\mathbf{x})$ and to the left (−) of $g_2(\mathbf{x})$. The region corresponding to class B lies either to the left or to the right of both lines.

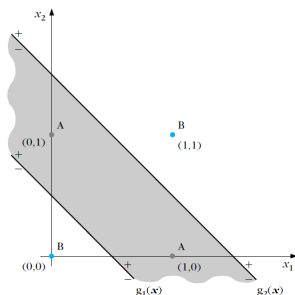


Figure: Decision lines realized by a two-layer perceptron

The two-layer perceptron

What we have really done is to attack the problem in **two successive phase**.

- During the **first phase**, we calculate the position of a feature vector $\mathbf{x}$ with respect to each of the two decision lines.
- In the **second phase**, we combine the results of the previous phase and we find the position of $\mathbf{x}$ with respect to both lines.

First phase

Realization of the two decision lines (**hyperplanes**), $g_1(\cdot)$ and $g_2(\cdot)$, during the **first phase** of computations is achieved with the adoption of two perceptrons with inputs x_1 , x_2 and appropriate synaptic weights.

The corresponding outputs are $y_i = f(g_i(\mathbf{x}))$, $i = 1, 2$, where the activation function $f(\cdot)$ is the step function with levels 0 and 1.

These are nothing else than the relative positions of the input vector with respect to each of the two lines. The computations during the first phase **perform a mapping** of the input vector $\mathbf{x}$ to a new one $y = (y_1, y_2)^T$.

1st Phase				2nd Phase
x_1	x_2	y_1	y_2	
0	0	0 (-)	0 (-)	B (0)
0	1	1 (+)	0 (-)	A (1)
1	0	1 (+)	0 (-)	A (1)
1	1	1 (+)	1 (+)	B (0)

Figure: Truth Table for the Two Computation Phases .

Second phase

The decision during the second phase is now based on the **transformed data**; that is, our goal is now to separate $(y_1, y_2) = (0, 0)$ and $(y_1, y_2) = (1, 1)$, which correspond to class *B* vectors, from the $(y_1, y_2) = (1, 0)$ which corresponds to class *A* vectors.

In other words, the **mapping** of the first phase **transforms the nonlinearly separable problem** to a **linearly separable** one.

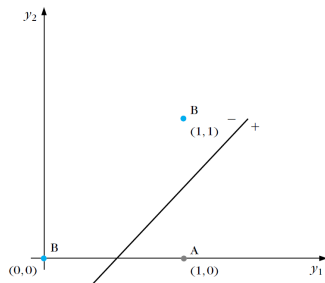


Figure: Decision line formed by the neuron of the second layer.

Two-layer perceptron

Each of the three lines is realized via a neuron with appropriate synaptic weights. The resulting **multilayer architecture** can be considered as a generalization of the perceptron, and it is known as a **two-layer perceptron** or a **two-layer feedforward neural network** .

The two neurons (nodes) of the first layer perform computations of the first phase and they constitute the so-called **hidden layer**. The single neuron of the second layer performs the computations of the final phase and constitutes the **output layer**

Two-Layer Perceptron

The action of the neurons of the hidden layer is actually a **mapping** of the input space $\mathbf{x}$ onto the vertices of a square of unit side length in the two-dimensional space.

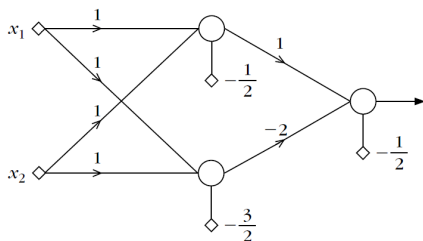


Figure: A two-layer perceptron solving the XOR problem..

Two-Layer Perceptron

For the more general case, we will consider input vectors in the l -dimensional space, that is, $\mathbf{x} \in \mathbb{R}^l$, and p neurons in the hidden layer.

Again employing the step activation function, the mapping of the input space, performed by the hidden layer, is now onto the vertices of the **hypercube of unit side length** in the p -dimensional space, denoted by H_p . This is defined as

$$H_p = \{(y_1, \dots, y_p)^T \in \mathbb{R}^p, y_i \in [0, 1], 1 \leq i \leq p\}$$

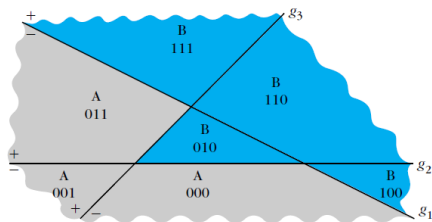
The vertices of the hypercube are all the points $\{(y_1, \dots, y_p)^T$ of H_p with $y_i \in \{0, 1\}$, $1 \leq i \leq p$.

Two-Layer Perceptron

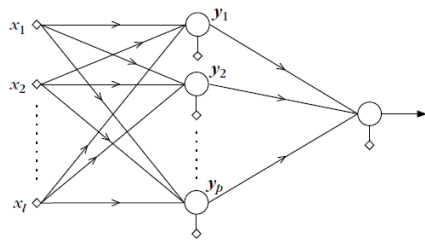
The **mapping of the input space** onto the vertices of the hypercube is achieved via the **creation of p hyperplanes**.

Each of the **hyperplanes** is created **by a neuron** in the hidden layer, and the output of each neuron is 0 or 1, depending on the relevant position of the input vector with respect to the corresponding hyperplane.

Example



(a) A two-layer perceptron



(b) Polyhedra formed by the neurons of the first hidden layer of a multilayer perceptron.

Example

The **first layer** of neurons divides the input I -dimensional space into **polyhedra**, which are formed by **hyperplane intersections**. All vectors located within one of these polyhedral regions are mapped onto a specific vertex of the unit H_p hypercube.

The **output neuron** subsequently realizes another hyperplane, which separates the hypercube into two parts, having some of its vertices on one and some on the other side. This neuron provides the multilayer perceptron with the potential to classify vectors into classes consisting of **unions of the polyhedral regions**.

A two-layer perceptron can separate classes each consisting of **unions of polyhedral regions but not any union of such regions**. It all depends on the relative positions of the vertices of H_p , where the classes are mapped, and on whether or not these are linearly separable.

Three-layer perceptrons

The inability of the two-layer perceptrons to separate classes resulting from **any union of polyhedral regions** springs from the fact that the output neuron can realize only a single hyperplane. This is the same situation confronting the basic perceptron when dealing with the XOR problem. The difficulty was overcome by constructing two lines instead of one.

Three-layer perceptrons

- Let us assume that all regions of interest are formed by intersections of p l -dimensional half-spaces defined by the p hyperplanes.
- These are realized by the p neurons of the **first hidden layer**, which also perform the mapping of the input space onto the vertices of the H_p hypercube of unit side length.
- In the sequel let us assume that class A consists of the union of K of the resulting polyhedra and class B of the rest.
- We then use K neurons in the **second hidden layer**. Each of these neurons realizes a hyperplane in the p -dimensional space.

Three-layer perceptrons

- The **synaptic weights** for each of the second-layer neurons are chosen so that the realized hyperplane leaves only one of the H_p vertices on one side and all the rest on the other.
- Each time an input vector from class A enters the network, one of the K neurons of the second layer results in a 1 and the remaining $k - 1$ give 0.
- In contrast, for class B vectors all neurons in the second layer output a 0.
- Classification is now a straightforward task. Choose the output layer neuron to realize an OR gate. Its output will be 1 for class A and 0 for class B vectors.

Three-layer perceptrons

The number of neurons in the **second hidden layer** can be reduced by exploiting the geometry that results from each specific problem—for example, whenever two of the K vertices are located in a way that makes them separable from the rest, using a single hyperplane.

In summary, the neurons of the **first layer** form the hyperplanes, those of the **second layer** form the regions, and finally the neurons of the **output layer** form the classes.

References

- Theodoridis, S. & Koutroumbas, K. *Pattern Recognition*, 4th ed., Academic Press, 2009